Autonomous Debugging Agent with an Agentic Loop

Development, Architecture and
Comparative Experimental Evaluation

Local LLM • Python • Ollama • pytest • AST Validation

Author: Danai Chanlaridou

Duration: October - December 2026

Documentation Type: Technical Report

Contents

1 Abstract **2**

2 Introduction and Objectives **2**

3 Development Environment and Setup **2**
3.1 Host hardware and software . . . 2
3.2 Installation and verification . . . 3
3.3 Initial local-model benchmark . . . 3

4 System Architecture **3**
4.1 Core agentic loop . . . 4
4.2 Patch acceptance and recovery . . . 4
4.3 Target identification and dependencies . . . 4

5 Implementation History **5**

6 Experimental Methodology **5**
6.1 Experimental design . . . 5
6.2 Evaluation definitions . . . 6

7 Detailed Experimental Results **6**
7.1 Experiment 1: Multiple logical errors . . . 6
7.2 Experiment 2: Edge cases . . . 8
7.3 Experiment 3: Recursive functions . . . 10
7.4 Experiment 4: Multiple interacting functions . . . 13
7.5 Experiment 5: Binary-search algorithm . . . 15
7.6 Experiment 6: Stateful class . . . 16

8 Comparative Evaluation **18**
8.1 What the experiments demonstrate . . . 18
8.2 What the experiments do not demonstrate . . . 18
8.3 Additional analysis: controller versus model failures . . . 18

9 Security, Validity and Limitations **18**
9.1 Security boundary . . . 18
9.2 Internal and external validity . . . 19
9.3 Practical constraints . . . 19

10 Reproducibility and Recommended Evaluation Protocol **19**
10.1 Recorded execution commands . . . 19
10.2 Proposed fair benchmark . . . 19

11 Future Development **19**

12 Conclusion **20**

13 References **21**

A Simple Initial Demonstration **21**

1 Abstract

This project develops a local, test-guided autonomous debugging agent for Python programs. A small code-specialized language model, **Qwen2.5-Coder 1.5B**, is executed through Ollama. The agent runs a pytest suite, interprets failures, requests a repair, validates the proposed patch, and repeats the process until all tests pass or the attempt budget is exhausted. Later revisions introduce targeted function and method replacement, dependency-aware diagnosis, rejection of regressions, feedback within a run, structured logs, and rollback.

The development began with a simple addition-function demonstration and progressed to six defect scenarios. At least one successful run was documented for **five of the six scenarios**; the remaining scenario, involving interacting functions, was not solved in the recorded attempts. Thus, **5/6 (83.3%)** is a description of *scenario coverage across different agent revisions and selected runs*, not an estimated success probability of a single version. The evidence supports a functional proof of concept while highlighting substantial limitations in model reasoning, experimental comparability, and execution security.

2 Introduction and Objectives

Traditional debugging typically requires a developer to inspect errors, infer causes, modify code, and verify the result. An autonomous debugging agent attempts to automate part of this workflow using test failures as actionable feedback. Unlike a one-shot code generator, the agent is designed to *observe, act, check, and revise* over several iterations.

The primary objectives are: (i) implement a functioning local loop without paid model APIs; (ii) repair common Python defects using a small language model; (iii) prevent unacceptable patches from silently replacing working behavior; (iv) retain evidence of each run through logs and backups; and (v) determine which bug categories remain difficult under the selected resource constraints.

Scope. The documented system operates on controlled Python exercises and pre-existing pytest tests. It is not a general repository-level repair service, a formal verification system, or a secure platform for running untrusted code.

Agentic approaches combine reasoning with iterative actions and observations [7].

3 Development Environment and Setup

3.1 Host hardware and software

The initial development report records a Windows 10 Education laptop (version 21H2, build 19044.1387), 8 GB RAM, an NVIDIA GPU with 2 GB VRAM, driver 512.72, and reported CUDA 11.6. WSL 2 was present, but no Linux distribution was installed at the time of the checks. Subsequent experimental runs used **Python 3.11.2** inside a virtual environment with **pytest 9.2.0**. The active agent ran on Windows, without Docker isolation.

Table 1: Documented development stack.

Element	Configuration or purpose
Host	Windows 10; laptop with 8 GB RAM
Inference	Ollama with `qwen2.5-coder:1.5b`
Orchestration	Python 3.11.2 virtual environment
Test feedback	pytest (9.2.0 in recorded experiment checks)
Validation and logs	Python `ast`, `difflib`, `json`, `subprocess`, `pathlib` and related standard-library utilities
Execution isolation	No Docker sandbox; test processes execute on the host

The system uses Qwen2.5-Coder 1.5B [1], executed locally through Ollama [2].

3.2 Installation and verification

The following commands reproduce the documented Windows Command Prompt setup, assuming Python 3.11 and Ollama have already been installed:

```
ollama pull qwen2.5-coder:1.5b
ollama list
mkdir debugging-agent
cd debugging-agent
py -3.11 -m venv .venv
.venv\Scripts\activate.bat
python -m pip install ollama pytest
python -c "import ollama, pytest; print('Environment ready')"
```

For PowerShell sessions where activation scripts are restricted, the virtual environment can be used without activation:

```
.\.venv\Scripts\python.exe --version
.\.venv\Scripts\python.exe -m pytest --version
.\.venv\Scripts\python.exe agent.py test1
```

This interpreter distinction is consequential. An early Experiment 1 attempt invoked a system Python 3.14 interpreter that lacked pytest (`No module named pytest`). The missing dependency prevented meaningful evaluation of the proposed corrections. Switching to the virtual-environment interpreter corrected the *environment issue*; it was not a fix to the test program.

3.3 Initial local-model benchmark

The original report contains a small code-generation benchmark that requested a primality-testing function. The recorded generation speed was **18.86 tokens/s**, with a total response time of **21.79 s**. Ollama reported approximately **20% CPU / 80% GPU placement**, while an NVIDIA snapshot showed 696 MiB occupied out of 2048 MiB and 0% utilization at the instant captured. Placement percentages must not be interpreted as instantaneous CPU/GPU usage. This was a feasibility check, not a systematic latency benchmark; no distribution of run times or repeated trials was supplied.

4 System Architecture

Patch validation uses Python Abstract Syntax Trees [5], while functional verification relies on pytest [3].

4.1 Core agentic loop

Figure 1 represents the later test-guided, patch-validated workflow. The model does not decide by itself whether a patch is correct: a separate controller checks structure and compares test outcomes. Even so, passing a limited suite cannot prove general program correctness.

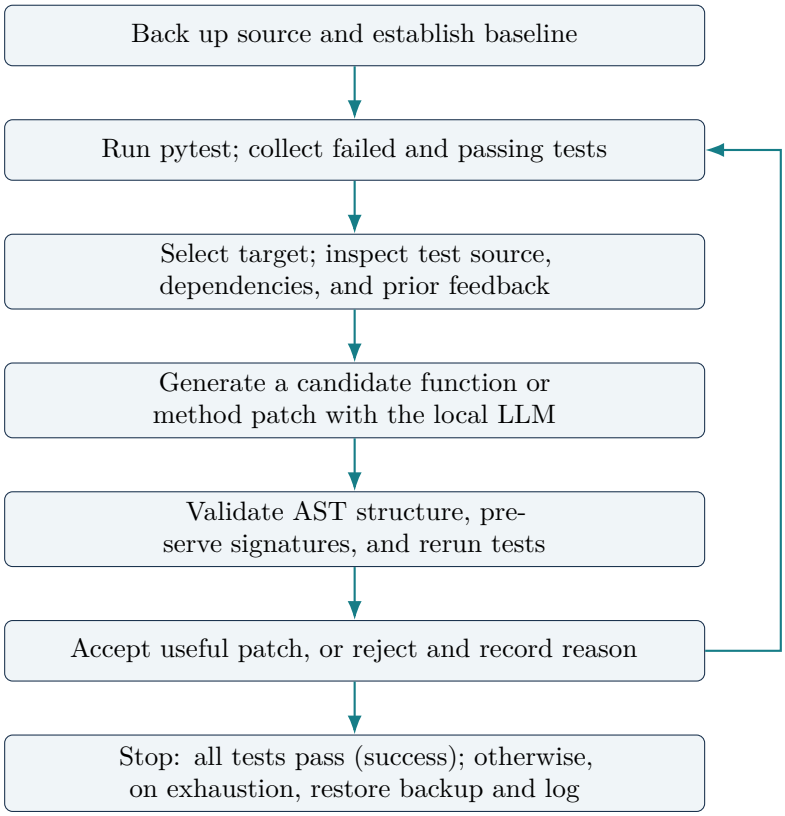


Figure 1: Generalized debugging loop, combining mechanisms introduced across later revisions. Not every historical version implements every stage.

4.2 Patch acceptance and recovery

A candidate patch can be rejected for invalid syntax, an incorrect output shape, missing or altered functions/signatures, no logical change, a duplicate attempt, or test regressions, depending on the version. These checks operate at different levels: `ast.parse()` establishes syntactic validity; structural checks preserve the expected program interface; pytest assesses observed behavior. **None of these checks isolates potentially malicious code.**

Backups store the pre-run source and allow restoration when the experiment fails. JSON logs record information about attempts, rejected patches, outcomes, and rollback. The "feedback memory" described here is *run-local*: it exposes earlier attempts to subsequent prompts within a debugging run, not knowledge learned persistently across executions.

4.3 Target identification and dependencies

Early revisions asked the LLM to rewrite broader source sections. Starting with V3.2, the controller increasingly focuses on one function at a time. Later dependency-aware variants inspect which functions call others; V3.7 extends AST targeting to class methods such as `BankAccount.deposit`. This improves the granularity of edits but also creates a target-selection problem: a failing high-level test may be caused by a dependency rather than by the function named in its failure.

5 Implementation History

The two source reports describe overlapping but distinct phases: an initial V1–V3 demonstration and a subsequent series of experiments on V3–V3.7. Table 2 preserves their chronology without treating every modification as an independently validated improvement.

Table 2: Version evolution documented in the supplied reports.

Version	Implemented mechanism or observation
V1	Basic loop: run pytest, request a model fix, replace code, and retest. No automatic preservation of the original source.
V2	Timestamped backup, rollback on unsuccessful completion, syntax checks, and JSON logging.
V3	Run-local feedback, patch validation, unified diffs, and richer logs; initial implementations attempted full-source repairs.
V3.1	Shorter pytest tracebacks and more explicit prompts, introduced after difficulty with recursion.
V3.2	Function-by-function debugging, AST-based structural checks, and cleaning of model output.
V3.3	Test-source-aware prompting, target selection informed by failures, and regression checks.
V3.4	Dependency analysis and dependency-aware prompts; observed repeated no-change proposals in Experiment 4.
V3.5	Multiple patch retries per iteration with failure-specific feedback; Experiment 4 still failed.
V3.6	More structured diagnosis, duplicate-attempt rejection, adaptive retries, and up to 7 iterations with 3 attempts each in the documented configuration.
V3.7	Class-aware AST analysis and method-level patching; successful `BankAccount` experiment.

Additional workflow illustration. The V3.5 revision proposed repeated patch-generation attempts inside each debugging iteration, followed by validation and feedback. The optional diagram below is included when the corresponding original image has been uploaded to the Overleaf project's `images/` directory.

Interpretive caution. Version labels identify evolving implementations, not a controlled ablation study. Some changes were made specifically after reviewing a failed scenario; therefore, improvement claims cannot be separated from task-specific tuning or the randomness of model output without further trials.

6 Experimental Methodology

6.1 Experimental design

The later study used six deliberately buggy scenarios covering independent logical errors, edge conditions, recursion, dependencies, algorithmic logic, and object state. Each scenario had a Python source file and a pytest test suite. The recorded process involved establishing failures, running an agent version, inspecting test outcomes, and restoring the original code when an attempt budget

was exhausted. The six experiments were not all run under one common frozen revision or one standardized number of model requests.

6.2 Evaluation definitions

For a run with N pytest cases, define its observed test pass fraction as

$$P_{\text{tests}} = \frac{\#\{\text{passing pytest cases}\}}{N}. \tag{1}$$

This is a run-specific observation, not a correctness proof. A *successful run* means all supplied tests pass under the agent's stopping condition. A *failed run* means the required tests did not all pass before the budget ended; the original code was then restored when rollback operated. An *unsupported run* means the controller could not represent or select the repair target, so model repair capability was not actually tested.

To summarize the historical records, we also report scenario coverage:

$$C_{\text{observed}} = \frac{\text{scenarios with at least one recorded successful run}}{\text{six selected scenarios}} = \frac{5}{6} \approx 83.3\%. \tag{2}$$

Because the numerator combines *different versions and the best observed attempts*, this quantity must not be interpreted as a generalization rate, a confidence estimate, or the success rate of V3.7.

7 Detailed Experimental Results

The experiments below are reconstructed from the supplied execution screenshots and written records. Code snippets distinguish *documented faulty expressions* from *illustrative corrected logic*; they are not verbatim copies of the final repository. An "iteration" is an agent loop step, whereas a "patch attempt" is one LLM-generated candidate: later versions may generate multiple attempts within one iteration. In every case, a green pytest result establishes success only for the assertions provided.

7.1 Experiment 1: Multiple logical errors

Purpose and initial defects

The first advanced experiment combined three *independent* logical mistakes in one Python module: `calculate_average` divided by one more than the list length; `find_maximum` initialized the maximum to zero and updated it in the wrong direction; and `is_even` returned the answer to the opposite parity condition.

Listing 1: Experiment 1: documented buggy expressions and intended corrections.

```
# Incorrect -> corrected
return total / (len(numbers) + 1) # -> total / len(numbers)
maximum = 0 # -> maximum = numbers[0]
if number < maximum: # -> if number > maximum:
return number % 2 == 1 # -> number % 2 == 0
```

For example, the average of $[2, 4, 6]$ must be 4, not 3; the maximum of an all-negative list must remain negative rather than default to zero; and `is_even(4)` must be `True`. The selected tests covered such behaviors, although exact coverage across all possible inputs was not independently audited.

Execution sequence and troubleshooting

1. In an early run, the model proposed logical corrections, but the agent could not assess them successfully. Diagnostic output later revealed `C:\Python314\python.exe: No module named pytest`. This was a *controller environment failure*, not evidence that the corrections were wrong. The unsuccessful run eventually restored the original source.
2. The interpreter was changed to `.venv\Scripts\python.exe`, which was Python 3.11.2 with pytest 9.2.0. This fixed the test execution environment.
3. On the documented successful V3 run, the first proposal changed the three faulty expressions. The next loop iteration ran pytest again and reported `SUCCESS: All tests passed!`; hence the recorded success took two loop iterations, but one corrective model proposal.
4. The accepted full-file response also inserted extraneous `test_*` functions into the implementation module. These did not prevent the supplied suite from passing, but they violated the intended scope of repair. Later versions therefore tightened patch constraints and shifted toward replacing individual functions.

Assessment. This experiment demonstrates repair of basic independent defects after the execution environment was corrected. It also shows why a passing suite alone cannot guarantee that an LLM respects edit boundaries.

```
PS C:\Users\Administrator\Desktop\debugging-agent> .\.venv\Scripts\python.exe -m pytest --version
pytest 9.2.0
PS C:\Users\Administrator\Desktop\debugging-agent> .\.venv\Scripts\python.exe agent.py test1
Running experiment: test1
Backup created: buggy_code_20261105_181156_176488.py

--- Iteration 1 ---
Tests failed. Asking LLM for a fix...

Proposed changes:
--- original
+++ proposed
@@ -1,18 +1,29 @@
-
 def calculate_average(numbers):
     total = 0
     for number in numbers:
         total += number
-    return total / (len(numbers) + 1)
+    return total / len(numbers)


 def find_maximum(numbers):
-    maximum = 0
+    maximum = numbers[0]  # Initialize maximum to the first element
     for number in numbers:
-        if number < maximum:
+        if number > maximum:
             maximum = number
     return maximum
```

Figure 2: Experiment 1: terminal diff from the successful run, including fixes to the average and maximum functions.

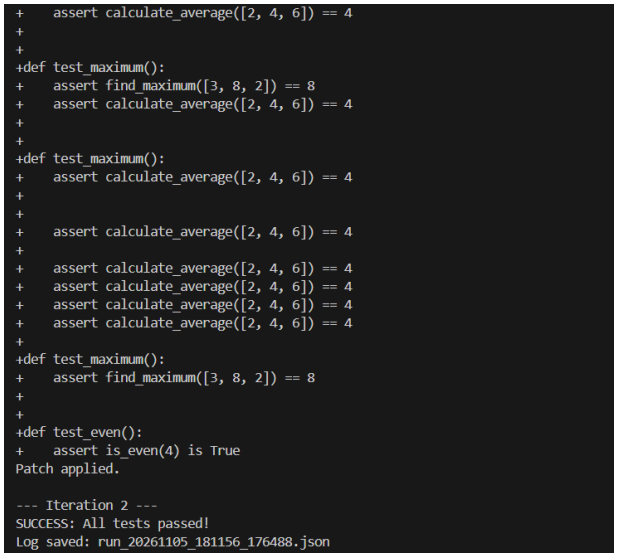


Figure 3: Experiment 1: additional test functions inserted into the implementation file; pytest eventually reports success. This is a patch-scope limitation, not a clean repair.

7.2 Experiment 2: Edge cases

Purpose and expected behavior

The second experiment targeted inputs often overlooked in normal arithmetic examples. A mean of an empty list had to return 0 according to this task's explicit contract; division by a zero denominator had to return `None`; and `clamp(value, minimum, maximum)` had to return the corresponding limit when the value lay outside the specified interval. These are task-specific expected behaviors, not universal requirements for every implementation of these functions.

Baseline failures and repair

Initial pytest output showed **three failing tests**: `test_average` raised `ZeroDivisionError` for an empty list; `test_safe_divide` raised `ZeroDivisionError` when $b = 0$; and `test_clamp` failed an assertion at a boundary because the buggy function returned a limit shifted by one. These are different failure types: the first two are exceptions, whereas the third produces a valid Python value that is semantically wrong.

The accepted patch used guard clauses before performing division and corrected the inclusive-boundary comparisons. The documented change was equivalent to:

Listing 2: Experiment 2: corrected behavior documented by the patch.

```
def calculate_average(numbers):
    if len(numbers) == 0:
        return 0
    return sum(numbers) / len(numbers)

def safe_divide(a, b):
    if b == 0:
        return None
    return a / b

def clamp(value, minimum, maximum):
    if value < minimum:
        return minimum
    if value > maximum:
        return maximum
```

```
    return value
```

This makes the equality case important: clamp$(0, 0, 10)$ must yield 0, not 1, and clamp$(10, 0, 10)$ must yield 10, not 9.

Observed outcome and interpretation

After the patch was applied in the first iteration, the second test execution reported **3/3 passing tests**. No further repair was needed. The result indicates that explicit edge-case assertions provided enough evidence for this small LLM to generate the corresponding guards and condition changes. It does not validate behavior for malformed limits, non-numeric values, or other cases outside the test contract.

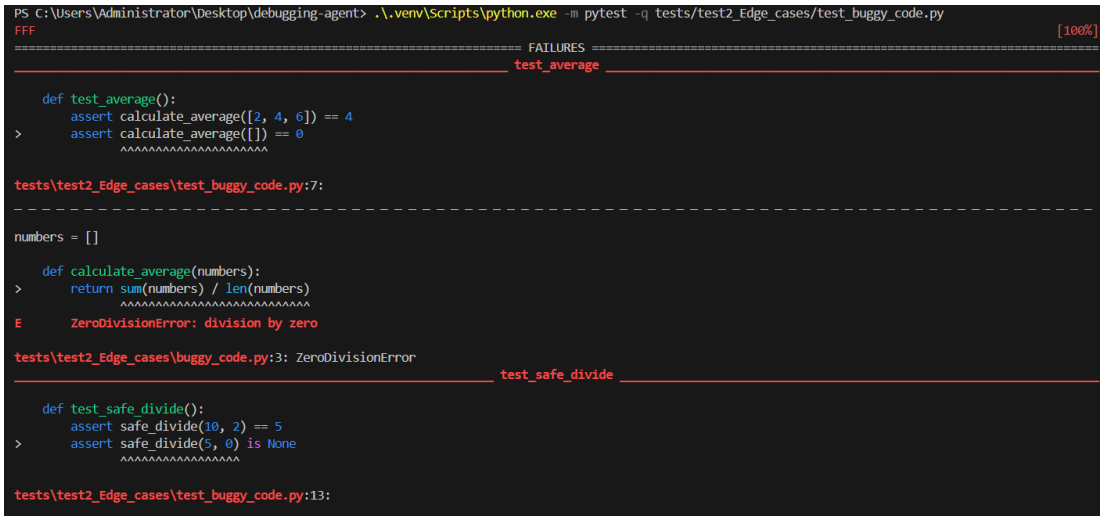


Figure 4: Experiment 2: initial pytest failures for the empty-list average and beginning of the zero-denominator test.

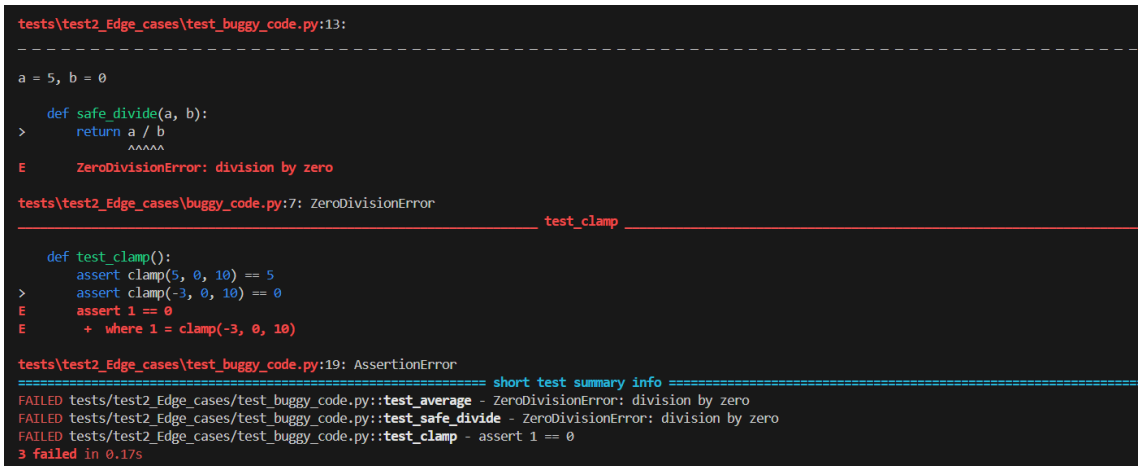


Figure 5: Experiment 2: baseline error output for division by zero and an incorrect clamp boundary.

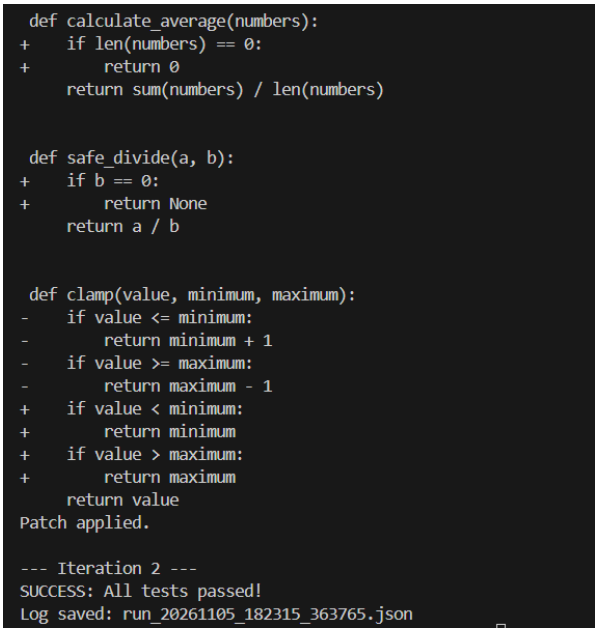


Figure 6: Experiment 2: accepted patch adds division guards, fixes clamping, and finishes with all tests passing.

7.3 Experiment 3: Recursive functions

Purpose and failure mechanisms

This experiment tested three different recursive defects. The faulty `factorial` lacked the correct termination behavior for the tested base case, leading to excessive recursion (`RecursionError`); `fibonacci` used the wrong recurrence and returned an incorrect number; and `recursive_sum` mishandled an empty list, producing an `IndexError`. Unlike the independent changes in Experiment 1, these errors depend on reasoning about base cases and how recursive calls approach them.

Correct behavior for the documented exercise is represented by:

Listing 3: Experiment 3: representative corrected recursive definitions.

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)

def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

def recursive_sum(numbers):
    if len(numbers) == 0:
        return 0
    return numbers[0] + recursive_sum(numbers[1:])
```

These examples explain the required logic for the tested non-negative inputs and lists; they do not establish a complete input-validation policy for negative factorial arguments, extreme recursion depth, or arbitrary data types. The crucial difference in the Fibonacci repair was changing `fibonacci(n - 3)` to `fibonacci(n - 2)`. For the recursive sum, indexing `numbers[0]` is safe only after an empty-list guard, and slicing must progress toward an empty sequence.

How the version changes affected the run

Table 3: Recorded trajectory in Experiment 3. The counts describe tests passing before rollback when the run failed.

Revision/run	Passed	Observation
V3	1/3	The factorial base case was eventually corrected, but Fibonacci and recursive sum remained faulty.
V3.1	1/3	Shorter error traces and prompt changes did not produce full repair; a syntactically valid but destructive model output exposed weak structural validation.
V3.2	1/3	Function-by-function edits targeted failures, but incomplete or unchanged patches remained.
V3.3, run 1	2/3	Both factorial and recursive sum were fixed; Fibonacci retained an incorrect recurrence.
V3.3, run 2	3/3	The same revision achieved complete test success on a later run.

In V3, the model repeated no-change proposals across several attempts before repairing the factorial base case. V3.1 shortened pytest output using options such as `-tb=short`, but the model also returned unusable output, including Markdown and an expression such as `obj['corrected_code']` in place of the program. The latter can pass a syntax parser while deleting required definitions, demonstrating why `ast.parse()` alone is insufficient. V3.2 introduced checks that preserve functions and narrowed the repair target, but no complete fix resulted.

V3.3 supplied the relevant *test source* in the prompt and guarded against regressions. In its first run, the agent repaired the sum and factorial and left Fibonacci returning the wrong value: the output recorded an assertion of `3 == 5` for the tested case. In the second run of the *same version*, the model eventually produced the needed correction, and pytest passed all three tests. This difference is important evidence of response variability, not proof that V3.3 always succeeds.

What the result demonstrates

The original failure and the corrected code have a clear causal relationship: a valid base case stops recursion, a correct recurrence computes Fibonacci, and a complete slice plus empty-list guard prevents invalid indexing. The gradual increase from 1/3 to 2/3, and eventually 3/3, suggests targeted test context may be helpful, but the documented runs cannot isolate its effect from stochastic generation or repeated opportunity.

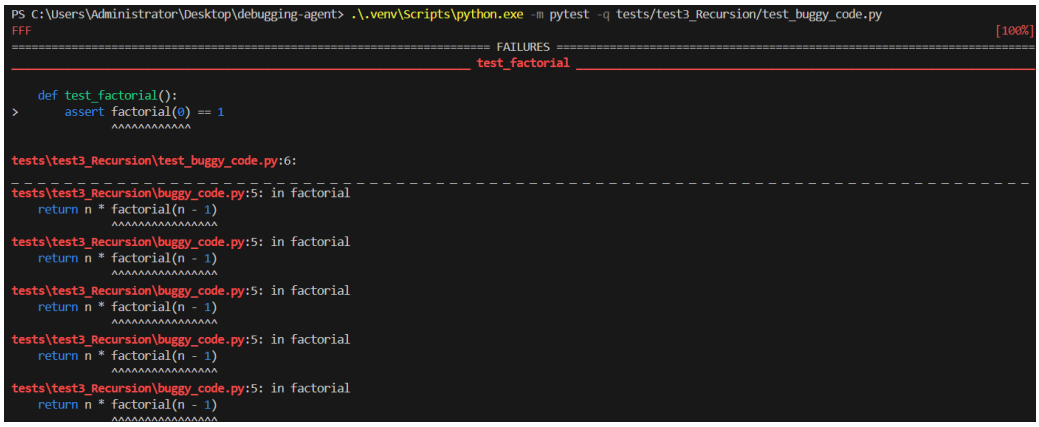


Figure 7: Experiment 3: initial failure of `factorial(0)`, with repeated recursive calls visible in pytest.

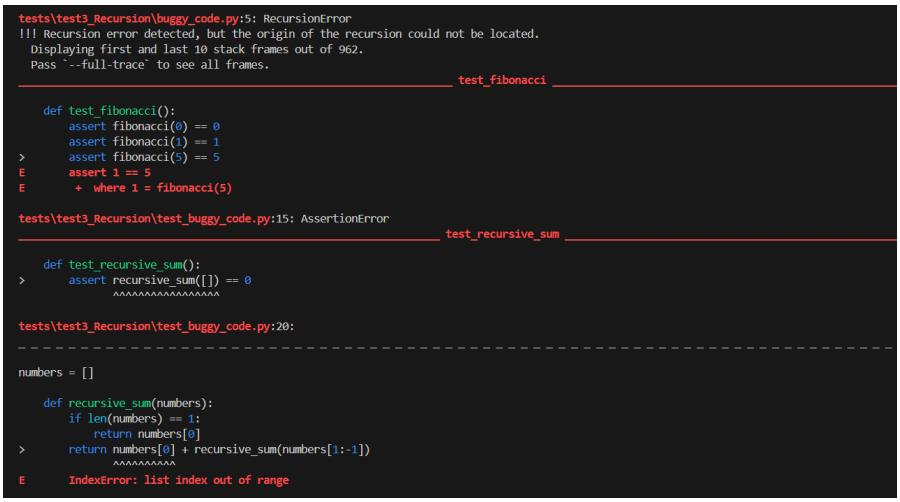


Figure 8: Experiment 3: Fibonacci assertion failure and `recursive_sum` invalid indexing at an empty list.

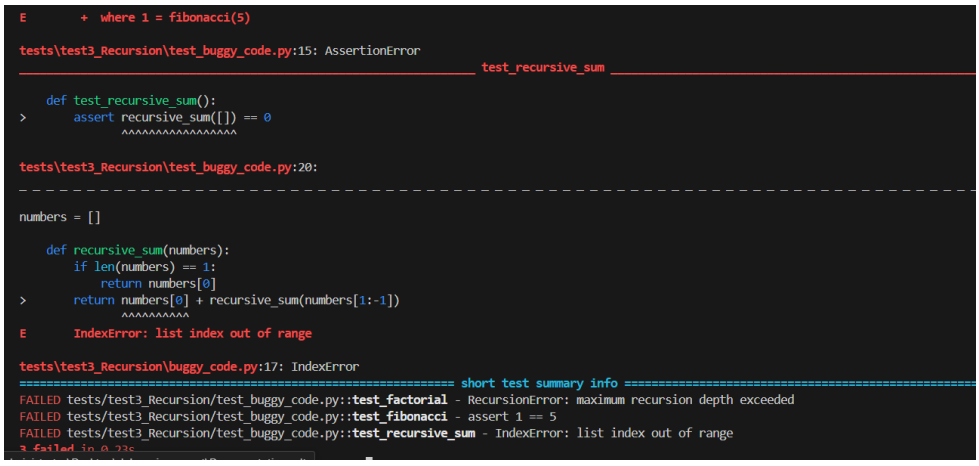


Figure 9: Experiment 3: pytest summary from an unsuccessful early recursion run; only one of the three tests passed.

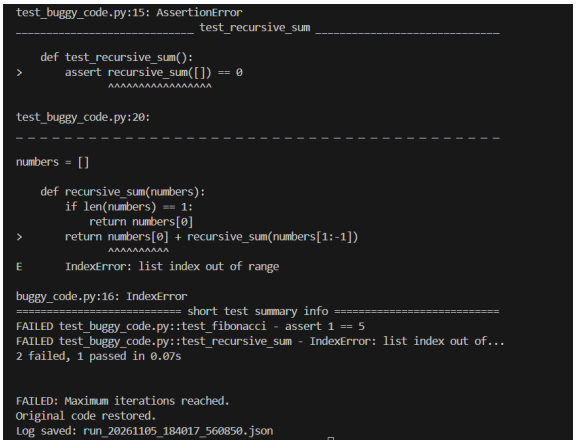


Figure 10: Experiment 3: further evidence of incomplete recursion repair and rollback after the iteration limit.

```
Target function: recursive_sum
Tests failed. Asking LLM for a fix...

Proposed changes:
--- original
+++ proposed
@@ -18,6 +18,6 @@


 def recursive_sum(numbers):
-    if len(numbers) == 1:
-        return numbers[0]
+    if len(numbers) == 0:
+        return 0
     return numbers[0] + recursive_sum(numbers[1:])
Patch applied.

SUCCESS: All tests passed!
Log saved: run_20261105_191828_863719.json
```

Figure 11: Experiment 3: successful V3.3 run showing the empty-list base-case fix for `recursive_sum` and final pytest success.

7.4 Experiment 4: Multiple interacting functions

Purpose: a dependency-sensitive calculation

This task introduced three interacting functions: `calculate_subtotal`, `apply_discount`, and `calculate_total`. Their correct relationship is

$$S = \sum_i p_i, \tag{3}$$

$$D = S(1 - d), \tag{4}$$

$$T = D(1 + t), \tag{5}$$

where p_i are item prices, d is the discount rate, and t is the tax rate. A recurring source of confusion was that a failure in `calculate_total` can be caused by earlier functions, and merely changing the final expression cannot repair a subtotal that omits an item.

Initial defects and concrete corrections

The documented buggy subtotal used slices such as `prices[:-1]`, excluding the last product; a later incorrect proposal used `prices[1:]`, excluding the first. Both violate $S = \sum_i p_i$. The discount function initially failed its test, and the higher-level total contained a tax calculation equivalent to `discounted + subtotal * tax`, taxing the full subtotal rather than the discounted amount. The target logic was:

Listing 4: Experiment 4: illustrative correct implementation of the documented calculation contract.

```
def calculate_subtotal(prices):
    return sum(prices)

def apply_discount(subtotal, discount):
    return subtotal * (1 - discount)

def calculate_total(prices, discount, tax):
    subtotal = calculate_subtotal(prices)
    discounted = apply_discount(subtotal, discount)
    return discounted * (1 + tax)
```

For the example $S = 100$, $d = 0.10$, $t = 0.20$, the discounted subtotal is 90 and the tax is 18. The required total is 108. The faulty calculation would yield $90 + 20 = 110$. Thus, both the value under test and the dependency chain must be inspected.

Attempt-by-attempt observations

Table 4: Observed failed Experiment 4 attempts by agent revision.

Revision	Best reported pass count	Important observation
V3.3	2/3	Subtotal and discount could be repaired, but the final total remained wrong. One proposed patch returned several functions when only one target was allowed.
V3.4	1/3	Dependency context was added; most proposals made no logical change.
V3.5	1/3	Up to three patch attempts per iteration and targeted rejection feedback did not resolve the scenario.
V3.6	1/3	Seven iterations and up to 21 candidates; repeated incorrect list-slice variants and eventual rollback.

One V3.3 trace records `calculate_subtotal` fixed early, an incomplete `apply_discount` change, and repeated attempts at `calculate_total`. At its last iteration, the model returned three function definitions instead of the requested single replacement. The AST validator rejected the output, leaving one failed test. A second V3.3 run likewise stopped with only 2/3 tests passing. Later refinements added dependency-aware prompts and repeated retries, but the model frequently reproduced unchanged code or incorrect slices. V3.6 sometimes concentrated its budget on the same failing prerequisite rather than revisiting the unresolved total.

Final result: failed. No documented revision completed all three tests in this scenario; partial pass counts were transient and the failed runs restored the original code. The experiment identifies a concrete limit of the combination of target selection, limited model capacity, and test-guided local repair. More retries did not automatically translate into better debugging.

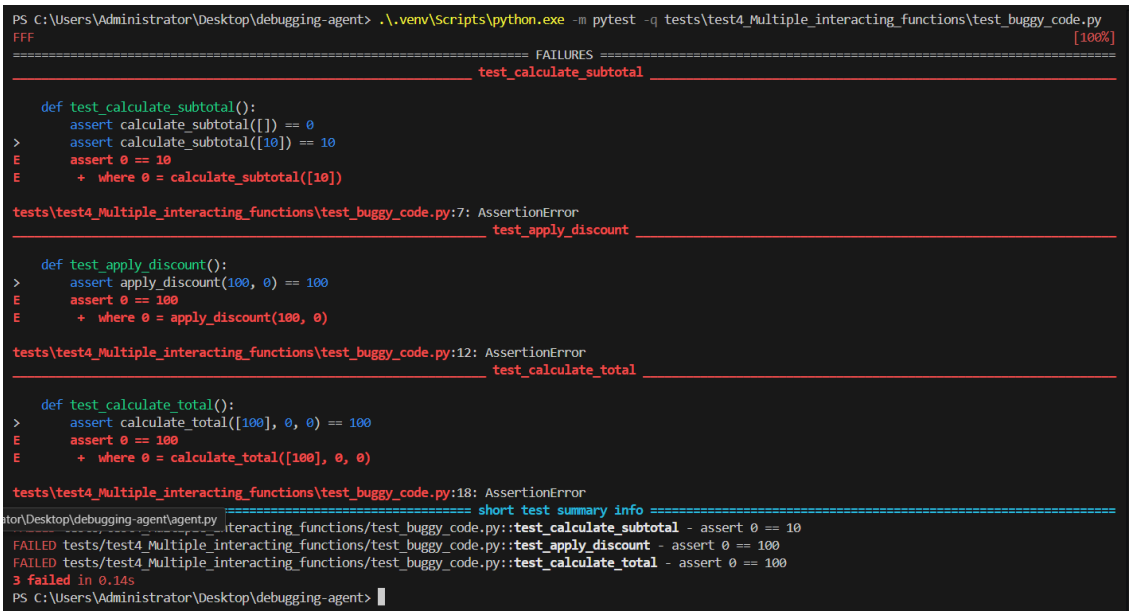


Figure 12: Experiment 4: baseline pytest output showing all three dependency-related tests failing before attempted repair.

7.5 Experiment 5: Binary-search algorithm

Bug and expected behavior

The fifth experiment examined a single algorithmic defect in binary search over an ordered list. When the middle element was smaller than the target, the buggy implementation narrowed the search toward the *left* half instead of the right half. This could omit the actual answer. For an ascending array, the update rules are:

Listing 5: Experiment 5: correct binary-search branch directions.

```
if arr[mid] == target:
    return mid
elif arr[mid] < target:
    left = mid + 1 # search the right half
else:
    right = mid - 1 # search the left half
```

The initial test run reported **2 failures and 4 passes**, including cases involving the first and last elements. The agent was then run with a *modified V3.6 script* (documented separately from the original V3.6 configuration). Its first proposed patch was accepted, and the run finished with **6/6 tests passing** in the first patch attempt.

Interpretation. Unlike the multi-function calculation in Experiment 4, this mistake is local to one decision branch, making its cause and correction easier to specify. The observed success is strong evidence for this particular task, but the modified script prevents treating the result as a controlled comparison of unmodified V3.6 across experiments.

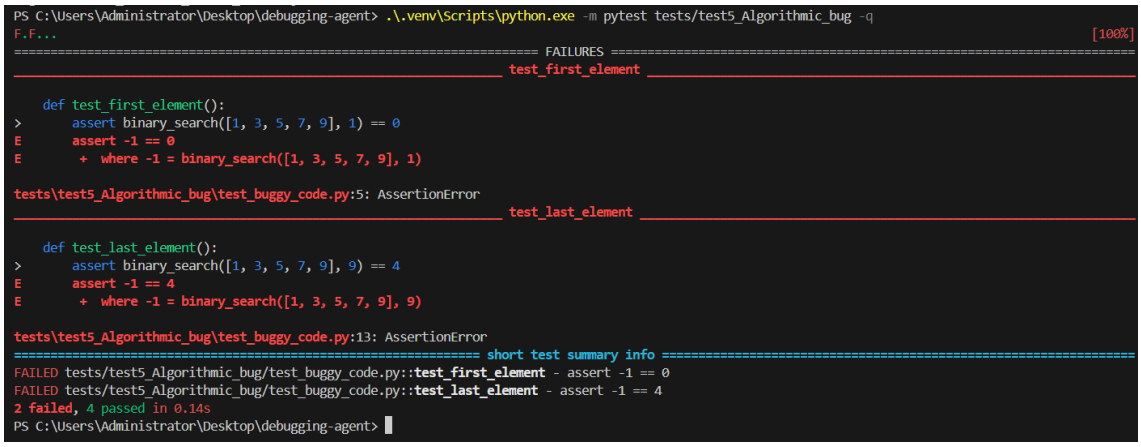


Figure 13: Experiment 5: baseline binary-search suite with 2 failures and 4 passes.

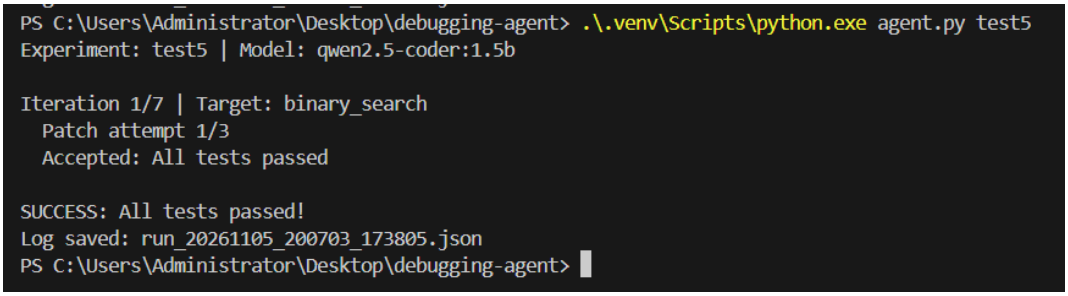


Figure 14: Experiment 5: first patch accepted by the modified V3.6 agent, followed by successful completion.

7.6 Experiment 6: Stateful class

Purpose and initial observed failures

A `BankAccount` class maintains a balance over sequences of operations. The intentionally faulty `deposit` replaced the existing balance with the deposited amount, and `withdraw` added the amount rather than subtracting it. The desired semantics for successful, valid operations can be summarized as:

Listing 6: Experiment 6: illustrative corrected state transitions (not a verbatim source patch).

```
# On a valid deposit:
self.balance += amount

# On a permitted withdrawal:
self.balance -= amount
```

This scenario is harder than fixing a pure arithmetic function because the outcome of each method is visible through later operations and through the `get_balance` accessor. The documented baseline had **3 failed and 2 passed** tests. Failures included a withdrawal leaving 140 instead of 60 and a multiple-operation sequence leaving 80 instead of 120, as shown in the screenshots. The corresponding tests also checked initial balance and insufficient-funds behavior.

Why V3.6 could not start the repair

The older AST controller identified top-level functions only. When it encountered a target such as `BankAccount.deposit`, it printed `No matching top-level function found`, restored the original code, and saved a log. Because no usable method patch was generated, this is best classified as *unsupported target representation* rather than a failure of the LLM to infer the appropriate deposit operation.

Changes in V3.7 and recorded debugging sequence

Version 3.7 extended the controller to inspect class AST nodes and replace a selected *method* rather than an entire class. Its trace documents:

1. **Iteration 1, `BankAccount.deposit`:** first candidate accepted with "Fewer failing tests." This is consistent with correcting the deposit state update.
2. **Iteration 2, `BankAccount.get_balance`:** the first candidate was rejected for regressions in `test_deposit`, `test_initial_balance`, and `test_insufficient_funds`. A second candidate was rejected because it did not return exactly one function definition; the third made no logical change. No harmful accessor patch was accepted.
3. **Iteration 3, `BankAccount.withdraw`:** first candidate was accepted with "All tests passed." The final suite reported **5/5 passing tests** and a JSON log was saved.

The method-level validator therefore did two useful things: it made previously unaddressable methods eligible for repair, and it prevented a proposed change that would break already-passing behavior. This does not prove correctness beyond the observed five assertions, and rejected regressions do not imply secure execution.

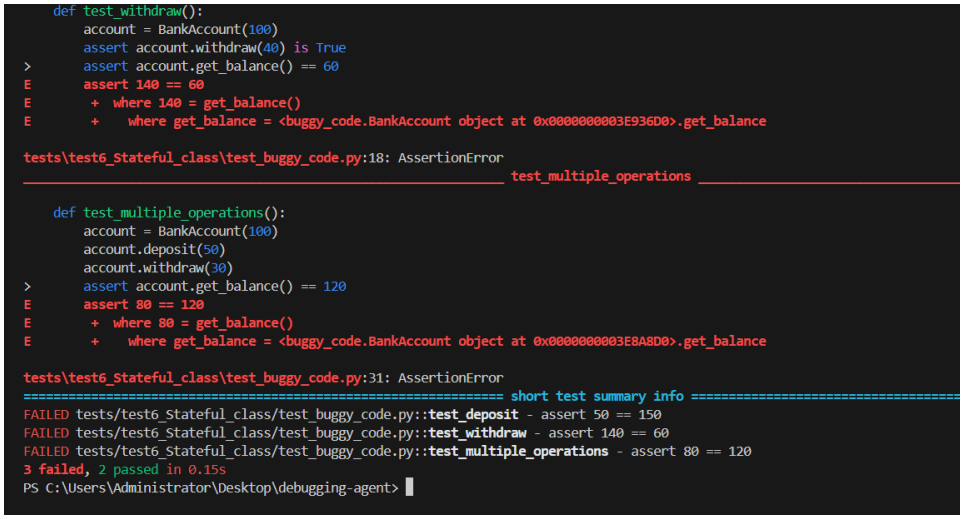


Figure 15: Experiment 6: initial pytest evidence of incorrect account balances after withdrawal and multiple operations.

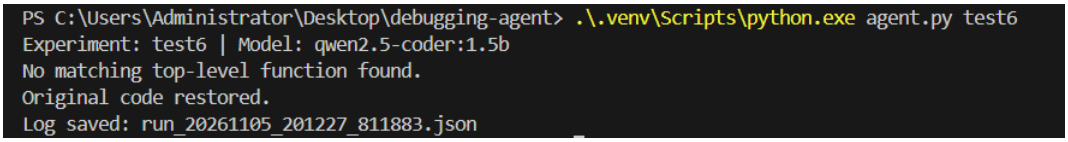


Figure 16: Experiment 6: V3.6 cannot identify a class method as a top-level function, so no repair is attempted.

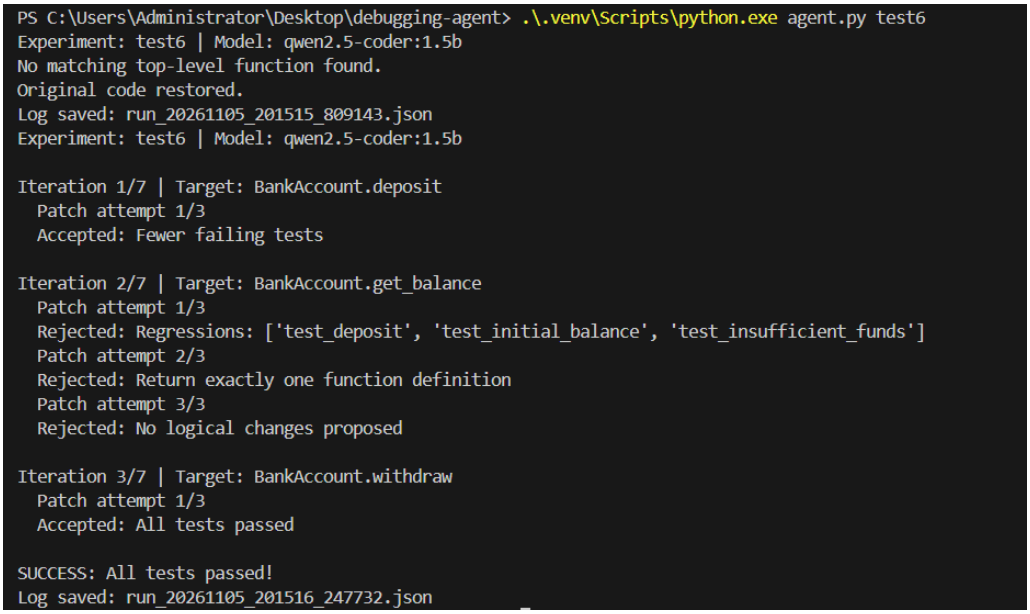


Figure 17: Experiment 6: V3.7 accepts deposit, rejects three problematic accessor patches, and completes the withdrawal repair with all tests passing.

8 Comparative Evaluation

Table 5: Best documented outcomes. Counts are not from a common final version.

Case	Defect class	Best recorded result	Version
1	Multiple independent logic errors	3 functions repaired; success	V3
2	Edge cases	3/3 tests; success	V3
3	Recursion	3/3 tests; second V3.3 run	V3.3
4	Interacting functions	2/3 intermediate; failed	V3.3
5	Binary search	6/6 tests; first patch	Modified V3.6
6	Stateful class	5/5 tests; 3 iterations	V3.7

8.1 What the experiments demonstrate

Three observations are reasonably supported by the provided records. First, a local agentic loop can repair selected Python bugs without a paid model API. Second, controller-side decisions matter: AST-based patch boundaries, regression tests, and rollback made the later experiments more disciplined. Third, the most persistent observed weakness was not merely syntax generation but repeated logically incorrect proposals in the presence of interacting functions.

8.2 What the experiments do not demonstrate

The reports do *not* establish a population-level repair success rate, statistically significant improvement from V3 to V3.7, or superior performance caused by any one feature. Several runs used different limits and revised code. Experiment 3 was retried with the same version; Experiment 5 used a modified version; Experiment 6 required new class support. Some intermediate pass counts were observed before a final rollback, so those partial changes were not retained as successful repairs.

8.3 Additional analysis: controller versus model failures

The results suggest two distinct sources of failure. *Controller failures* include launching the wrong Python interpreter, being unable to identify a class method, accepting syntactically valid but structurally destructive output, or selecting a misleading target. *Model-generation failures* include repeating unchanged source, proposing an incorrect recurrence, and misunderstanding tax/discount dependencies. Distinguishing these causes is essential: adding retries may help response variability, but it cannot repair an unsupported AST target; conversely, stronger structural validation does not make the model reason correctly.

9 Security, Validity and Limitations

9.1 Security boundary

The prototype invokes tests on the Windows host. Generated Python code and test modules can perform filesystem, process, or network operations. Static AST parsing and regression checks do *not* prevent such side effects. The current setup should be limited to controlled, trusted exercises; untrusted repositories require an isolated execution environment with explicit resource, filesystem, and network restrictions.

9.2 Internal and external validity

The evidence is based on a small hand-selected set of synthetic bugs. The tests were also used to guide iterative development, so the examples may have influenced the design. Different configurations, unequal repeated trials, and incomplete timing data prevent reliable comparison across versions. A test suite can miss defects not represented in its assertions; test success is therefore a conditional outcome rather than a proof of full correctness.

9.3 Practical constraints

The 1.5B model was chosen for limited local hardware and no paid inference costs. It can return incomplete functions, explanations rather than code, unchanged patches, and incorrect edits. The documented system does not yet support comprehensive multi-file repository analysis, formal semantic verification, persistent cross-run learning, or sandboxed execution.

10 Reproducibility and Recommended Evaluation Protocol

10.1 Recorded execution commands

From the project root, using the interpreter from the virtual environment:

```
.\.venv\Scripts\python.exe -m pytest --version
.\.venv\Scripts\python.exe agent.py test1
.\.venv\Scripts\python.exe agent.py test2
.\.venv\Scripts\python.exe agent.py test3
.\.venv\Scripts\python.exe agent.py test4
.\.venv\Scripts\python.exe agent.py test5
.\.venv\Scripts\python.exe agent.py test6
```

Each scenario must start from its original buggy implementation; a previous successful run otherwise changes the baseline. Archive the exact agent script, test suite, model tag, runtime configuration, and JSON logs associated with each result.

10.2 Proposed fair benchmark

A stronger follow-up study would freeze *one* agent revision and use identical test suites, model settings, stopping criteria, and hardware. Run multiple independent trials per scenario, restore the same baseline before every trial, and record (i) full-suite success/failure, (ii) failing-test trajectory, (iii) number of iterations and total patch proposals, (iv) rejected proposals by reason, (v) wall-clock time, and (vi) rollback behavior. Include unseen tasks not used to design prompts or choose features. A later comparison with a larger local model should vary the model while holding the agent and evaluation protocol constant.

Suggested summary metric. For scenario j, with r_j independent runs, report

$$\widehat{s}_j = \frac{1}{r_j} \sum_{k=1}^{r_j} \mathbf{1}\{\text{run } k \text{ passes all tests}\}.$$

Unlike the historical 5/6 coverage statistic, this metric measures repeated success under a fixed configuration; uncertainty intervals should accompany it when enough trials are available.

11 Future Development

Future evaluations could incorporate real-world software repair benchmarks, such as SWE-bench [6].

High-priority next steps are (1) execution isolation before accepting third-party code, (2) a frozen evaluation protocol with repeated runs and unseen test cases, and (3) stronger diagnosis of interacting functions, including traceable dependency selection. Additional directions include multi-file patching, controlled persistent memory, richer failure summaries, and explicit checks that reject modifications to test files or unrelated functions. These are *proposed* extensions, not capabilities demonstrated in the current evidence.

12 Conclusion

The Autonomous Debugging Agent demonstrates a working local agentic loop that integrates Ollama, Qwen2.5-Coder 1.5B, Python, and pytest. Across its evolution from V1 through V3.7, the controller gained backup and rollback, structured logging, function-level and method-level repair, and regression-sensitive patch evaluation. The documented successful experiments span independent logic errors, edge cases, recursion, binary search, and a stateful class. Repeated failure on interacting functions provides an equally important boundary condition.

The proper conclusion is therefore a **promising but constrained proof of concept**: the approach is technically feasible on modest local hardware and can solve selected tasks, while its reliability, security, and generalization remain unproven. A secure sandbox and controlled repeated benchmarking are essential before broader deployment claims can be made.

13 References

References

[1] B. Hui et al., *Qwen2.5-Coder Technical Report*, arXiv preprint arXiv:2409.12186, 2024. Available: https://arxiv.org/abs/2409.12186

[2] Ollama, *Ollama Documentation*. Available: https://docs.ollama.com/

[3] pytest Development Team, *pytest Documentation*. Available: https://docs.pytest.org/en/stable/

[4] Python Software Foundation, *Python 3 Documentation*. Available: https://docs.python.org/3/

[5] Python Software Foundation, *ast — Abstract Syntax Trees*. Available: https://docs.python.org/3/library/ast.html

[6] C. E. Jimenez et al., *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?*, International Conference on Learning Representations (ICLR), 2024. Available: https://arxiv.org/abs/2310.06770

[7] S. Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, International Conference on Learning Representations (ICLR), 2023. Available: https://arxiv.org/abs/2210.03629

A Simple Initial Demonstration

The first development report used a deliberately incorrect addition function:

```
# Initial program
def add(a, b):
    return a - b

# Expected repair
def add(a, b):
    return a + b
```

The corresponding three pytest cases checked positive, mixed-sign, and zero-input addition. The recorded agent execution proposed the correct operator change and the next test iteration passed. This is an end-to-end integration demonstration, distinct from the six later experiments.